Ders 28 Çalışma Özeti

Ders 28 Çalışma Özeti

Genel Konular

- Dersin tarihi 3 Ocak 2022 Pazartesi. Dönemin son kaydıdır, yeni yılın ilk dersidir. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:10'da başlar. 01:01-01:14 arası teneffüstür.
  - Slaytlar Silberschatz, *Operating System Concepts* (9. baskı) 8. bölüm: ana bellek (main memory).
- Ana bellek yönetiminin neden gerekli olduğu: çoklu programlama (multiprogramming)
- Temel donanım: yazmaçlar, ana bellek, önbellek; taban ve sınır yazmaçlarıyla koruma (base and limit registers)
- Adres bağlama (address binding): derleme, yükleme ve çalışma zamanı
- Mantıksal (logical) ve fiziksel (physical) adres; bellek yönetim birimi (memory management unit, MMU)
- Dinamik yükleme (dynamic loading), dinamik bağlama (dynamic linking) ve paylaşılan kütüphaneler
- Takas (swapping); mobil sistemlerde takasın yerine kullanılan yöntemler
- Bitişik bellek ayırma (contiguous allocation)
  - Sabit ve değişken bölümler; first fit, best fit, worst fit; dış parçalanma (external fragmentation); sıkıştırma (compaction)
- Bölümleme (segmentation)
- Final sınavı bilgilendirmesi

Hocanın Özellikle Vurguladığı Kısımlar

- **Final kapsamı:** Baştan sona işlenen her şey ve laboratuvarda yapılanlar dahil.
  - 8. bölümün derste işlenmeyen kısmından da sorumlusunuz. Sayfalama (paging), bellek koruma (memory protection) ve paylaşılan sayfalar (shared pages) dahil, yaklaşık 60. slayta kadar.
- **Final biçimi:** Vizeler gibi çoktan seçmeli test.
  - Final programı henüz açıklanmadı. Bölüm web sayfasını takip edin.
- **Soruları dikkatli okuyun.**
  - Sorular kelime oyunu üzerine kurulu değil. "Which of the following is" ya da "are" yazılması tek veya çok doğru cevap anlamına gelmez.
  - 2. vizede "hangisi değildir" sorusunu "hangisidir" diye okuyup tam tersini işaretleyen bir grup öğrenci vardı.
  - Hoca olumsuz soruları büyük harf ya da kalın yazmaya çalışıyor ama bazen atlayabiliyor.
  - Süre yeterli: 25 soru için 45 dakika. Acele etmeyin.
- Bellek koruma işini işletim sistemi yapar.
  - Taban ve sınır yazmaçlarını yalnız işletim sistemi, ayrıcalıklı kipte (privileged mode) değiştirebilir.
- Takas istenen bir şey değildir.
  - Sabit disk de SSD de bellekten çok yavaştır. SSD'nin yazma ömrü sınırlıdır.

Kısa Tekrar Notları

- İşlemci doğrudan yalnız yazmaçlara ve ana belleğe erişir. Diske doğrudan erişemez.

- Bellek erişimi çok saat çevrimi sürer, işlemci bekler (stall). Önbellek bu beklemeyi azaltır.
- Koruma: üretilen adres a için $base \leq a < base + limit$ olmalı. Değilse donanım tuzağa (trap) düşer.
  - Örnek: base = 300040, limit = 120900. Geçerli aralık 300040-420939.
- Adres bağlama zamanları:
  - Derleme zamanı: yer biliniyorsa mutlak kod (absolute code).
  - Yükleme zamanı: yer derlemede bilinmiyorsa yeniden yerleştirilebilir kod (relocatable code).
  - Çalışma zamanı: süreç çalışırken yer değiştirebiliyorsa. Donanım (MMU) desteği gerekir.
- Mantıksal adres = CPU'nun ürettiği adres (sanal adres). Fiziksel adres = bellek biriminin gördüğü adres. Eşlemeyi MMU yapar.
- En basit MMU: yeniden yerleştirme yazmacı (relocation register). Fiziksel adres = mantıksal adres + yazmaç değeri.
- Dinamik yükleme: bir rutin, çağrılana kadar belleğe alınmaz. Programcının işidir.
- Dinamik bağlama: bağlama çalışma zamanına ertelenir. Bir saplama kodu (stub) rutini bulur. Paylaşılan kütüphaneler bellekte tek kopya tutulur.
- Takas: bir süreç bellekten yedek depolamaya (backing store) alınır, sonra geri getirilir. 100 MB'lık süreç, 50 MB/s aktarımla: çıkarma 2 s, geri getirme 2 s, toplam 4 s.
- Bitişik ayırmada boş alan seçimi:
  - First fit: yetecek ilk boşluk.
  - Best fit: yetecek en küçük boşluk.
  - Worst fit: en büyük boşluk.
- %50 kuralı: N ayrılmış blok için yaklaşık $0{,}5N$ blok parçalanmaya gider. Belleğin yaklaşık $1/3$'ü kullanılamaz.
- Bölümleme: mantıksal adres = ⟨segment numarası, ofset⟩. Segment tablosu her segment için taban ve sınır tutar.

Detaylı Açıklamalar

Neden bellek yönetimi?

Bellekte tek program varken iş kolaydır. CPU kullanımını artırmak için çoklu programlama yapılır: bellekte birden fazla süreç bulunur. Bu süreçlerin birbirinin alanına taşmadan çalışması gerekir.

CPU komutları getirir (fetch), çözer (decode) ve çalıştırır (execute). Operand bellekteyse bellekten getirilir. Sonuç yazmaçta ya da bellekte saklanır. Bellek birimi yalnız bir adres akışı görür. Bu adresi kimin ürettiğini bilmez.

Yazmaç erişimi genelde bir saat çevriminde biter. Bellek erişimi ise çok daha uzun sürer ve işlemci bekler. Önbellek, sık erişilen veriyi işlemciye yakın tutar. Önbelleği işlemci yönetir.

Önbellek, CPU'da çalışan sürecin erişimleriyle dolar. Bağlam değişiminden sonra yeni süreç farklı adreslere eriştiği için önbellek ıskaları (cache miss) artar. Eski süreç uzun süre sonra dönerse, verisi önbellekten çoktan atılmış olur (ör. LRU ile).

Taban ve sınır yazmaçlarıyla koruma

Her sürecin bellekte kendine ayrılmış bir bölgesi olmalıdır. Taban yazmacı bölgenin başını, sınır yazmacı boyunu tutar. Kullanıcı kipinde üretilen her adres donanımca kontrol edilir:

- Adres tabandan küçükse ya da taban + sınırdan büyük veya eşitse, işletim sistemine tuzak (trap) oluşur.
- İşletim sistemi bunu ölümcül hata (fatal error) olarak ele alır ve süreç sonlandırılır.

Taban ve sınır yazmaçlarını yalnız işletim sistemi, çekirdek kipinde yükleyebilir. Süreç yer değiştirdikçe bu değerler dinamik olarak güncellenir.

Adres bağlama

Kaynak kod derleyiciden (ya da assembler'dan) geçer ve nesne koda (object code) dönüşür. Bağlayıcı (linker) başka modülleri ekler. Yükleyici (loader) programı belleğe alır. Bazı kütüphaneler çalışma sırasında dinamik olarak bağlanır.

Derleme sırasında değişkenlerin bellekteki yeri genelde bilinmez. Bu yüzden semboller kullanılır. Semboller, gerçek adreslere bağlama, yükleme ya da çalışma sırasında bağlanır.

- Derleme zamanında bağlama: Süreç belleğe hep aynı yere konacaksa. Bazı mikrodenetleyiciler hâlâ böyle çalışır.
- Yükleme zamanında bağlama: Kod yeniden yerleştirilebilir üretilir. Yüklendiği adrese göre adresler hesaplanır.
- Çalışma zamanında bağlama: Takas gibi işlemler yüzünden süreç çalışırken başka adrese taşınabiliyorsa gerekir. Bugünkü sistemlerin çoğu böyle çalışır ve donanım desteği gerekir.

Örnek: Program kendi adres alanının 300. adresine erişmek istiyor. Ama süreç bellekte 15000. adresten başlıyor. Erişim 15300. fiziksel adrese yönlendirilmelidir.

Slayttaki MMU örneğinde yeniden yerleştirme yazmacı 14000, mantıksal adres 346'dır. Fiziksel adres $14000 + 346 = 14346$ olur.

Dinamik yükleme ve dinamik bağlama

- **Dinamik yükleme:** Hata işleme gibi az çalışan rutinler baştan belleğe alınmaz. Diskte yeniden yerleştirilebilir biçimde bekler. Çağrılınca belleğe getirilir. Bu işletim sisteminin değil, uygulama geliştiricinin işidir. İşletim sistemi en fazla kendi kütüphanelerini bu şekilde sunabilir.
- **Statik bağlama:** Kütüphane kodu yükleme öncesinde programın içine gömülür. Her programda ayrı bir kopya bulunur.
- **Dinamik bağlama:** Bağlama, programın süreç olarak çalıştığı ana ertelenir. Kütüphane çağrısının yerine küçük bir saplama (stub) konur. Saplama çalışınca rutinin adresini bulur, rutin bellekte yoksa belleğe getirir.
  - Kütüphanenin tek kopyası bütün süreçlerce kullanılır: paylaşılan kütüphane.
  - Kütüphanenin geriye uyumsuz birden fazla sürümü varsa, her sürümden birer kopya tutulur. Bu yine her uygulamaya ayrı kopya tutmaktan iyidir.

Takas

Bellekte yer yokken bir süreç seçilir. Bellek görüntüsü yedek depolamaya (backing store) yazılır (swap out). Açılan yere yeni süreç alınır. Hazır kuyruğunda artık hem bellekteki hem de yedek depolamadaki süreçler bulunabilir. Zamanlayıcı diskteki bir süreci seçerse, süreç önce belleğe getirilir (swap in). Bu da bağlam değişimi süresini çok uzatır.

- 100 MB'lık süreç, 50 MB/s aktarım hızı: çıkarma 2 s, geri getirme 2 s, toplam 4 s.
- SSD'ler 550-1000 MB/s civarına çıksa da yazma ömürleri sınırlıdır. Sürekli takas, kullanıcı verisi olmayan yazmalarla diskin ömrünü tüketir.
- Günümüzde Linux ve Windows takasın değiştirilmiş bir hâlini kullanır. Takas normalde kapalıdır. Bellek kullanımı bir eşiği (ör. %80-90) geçince açılır, belli bir eşiğin altına düşünce kapanır.
- G/Ç bekleyen süreç takas edilmemelidir. G/Ç bitince aygıt verisi sürecin bellek alanına yazılmak ister; süreç bellekte değilse veri yanlış yere gider. Çözümler:
  - G/Ç bekleyen süreci takas etmemek.
  - G/Ç'yi çekirdek tamponlarına yapıp süreç dönünce kopyalamak (double buffering). İki kez kopyalama olduğu için ek yük getirir.

Mobil sistemler takası desteklemez. Flash belleğin hem boyutu hem yazma sayısı sınırlıdır.

- iOS bellek azalınca uygulamalardan bellek bırakmalarını ister. Kod gibi salt okunur veriler bırakılır, gerektiğinde flash'tan tekrar okunur. Yeterli bellek açılmazsa uygulama sonlandırılır.
- Android benzer davranır. Ama uygulamayı sonlandırmadan önce durumunu (state) flash'a yazar. Uygulama kaldığı yerden devam edebilir.

Bitişik bellek ayırma

Bellek iki kısma ayrılır: işletim sistemi ve kullanıcı süreçleri. İşletim sistemi genelde kesme vektörünün bulunduğu uçta tutulur. Her süreç bitişik tek bir bölgede bulunur. Koruma, yeniden yerleştirme yazmacı ve sınır yazmacıyla sağlanır: mantıksal adres sınırdan küçükse yeniden yerleştirme yazmacıyla toplanır, değilse tuzağa düşülür.

Sabit bölümler: Bölüm boyu sistemdeki en büyük çalıştırılabilir dosyaya göre seçilir. Küçük süreçler bölümün büyük kısmını boş bırakır. Bellekte tutulabilecek süreç sayısı azalır. IBM OS/360 bu yapıyı kullanmış, sonra değişken bölümlere geçmiştir.

Değişken bölümler: Her sürece ihtiyacı kadar yer verilir. Süreçler girip çıktıkça bellekte boşluklar (hole) oluşur. Bir süreç, iki ayrı boşluğun toplamına sığsa bile, bitişik olmadıkları için yerleştirilemeyebilir.

Boşluk seçme stratejileri:

- **First fit:** Yetecek ilk boşluk seçilir. Aramaya her seferinde baştan değil, son kalınan yerden de başlanabilir.
- **Best fit:** Yetecek en küçük boşluk seçilir. Bütün liste taranmalıdır (liste boyuna göre sıralı değilse).
- **Worst fit:** En büyük boşluk seçilir. Fikir, geriye kalan boşluğun başka bir sürece yetecek kadar büyük kalmasıdır.

Simülasyonlara göre first fit ve best fit, worst fit'ten hem hız hem bellek kullanımı açısından iyidir. First fit ile best fit arasında bellek kullanımında net bir üstünlük yoktur, ama first fit genelde daha hızlıdır.

Dış parçalanma: Toplam boş alan yeterli ama bitişik değil. %50 kuralı: N ayrılmış blok için yaklaşık $0{,}5N$ blok parçalanmaya gider. Kullanılamayan oran:

$$\frac{0{,}5N}{N + 0{,}5N} = \frac{1}{3}$$

Sıkıştırma (compaction): Süreçleri kaydırıp boşlukları tek bir büyük boşlukta toplamak. Yalnız yeniden yerleştirme çalışma zamanında yapılıyorsa mümkündür. Derleme ya da yükleme zamanında bağlanmış kod taşınamaz. Taşıma işleminin kendisi de maliyetlidir.

Dış parçalanmanın diğer çözümü, bitişiklik şartını kaldırmaktır. Süreç parçalara bölünür, parçalar bellekte farklı yerlere konur. Bölümleme ve sayfalama bu fikre dayanır.

Bölümleme

Programcı belleği doğrusal bir adres dizisi olarak değil, mantıksal parçalar olarak görür: ana program, prosedürler, fonksiyonlar, nesneler, yığın, diziler. Bölümleme bu bakışı destekler.

- Mantıksal adres iki parçadır: ⟨segment numarası, ofset⟩.
- Segment tablosu her segment için taban (fiziksel başlangıç adresi) ve sınır (segment boyu) tutar.
- Çevirme: Segment numarasıyla tablodan taban ve sınır okunur. Ofset sınırdan küçük değilse tuzağa düşülür. Küçükse fiziksel adres = taban + ofset.
- Segmentlere koruma bitleri verilebilir: okunur, yazılır, çalıştırılabilir. Kod segmenti çalıştırılabilir olmalı, veri segmentinin çalıştırılabilir olmasına gerek yoktur.

Intel 8086'da dört segment yazmacı vardır: CS (code segment), DS (data segment), SS (stack segment), ES (extra segment). MS-DOS bu yapı üzerinde çalışıyordu. Hoca, SI'nın bir segment yazmacı olmadığını, indeks yazmacı olduğunu belirtti.

Ders sonrası notlar

- Altyazıda "CS, stack segment" geçiyor. CS, code segment yazmacıdır; stack segment yazmacı SS'dir.
- 8086'da segment yazmacı, toplanan basit bir taban değeri değildir. Fiziksel adres $16 \times segment + ofset$ ile hesaplanır ve 20 bitliktir.

- %50 kuralının formülü derste "n artı bir bölü $2n$" diye karışık okundu. Doğru hesap: $0{,}5N/(N + 0{,}5N) = 1/3$.
- Takasın "eşik aşılınca açılması" anlatımı genel bir modeldir. Linux'ta takas sürekli etkindir; ne kadar takas yapılacağını `vm.swappiness` gibi ayarlar belirler.
- Altyazıdaki "threading'i doğrudan kullanmıyoruz" ifadesinin doğrusu "swapping'i doğrudan kullanmıyoruz"dur.
- Koruma kontrolündeki sınır koşulu: geçerli adresler $base \leq a < base + limit$ aralığındadır. Slayttaki örnekte son geçerli adres 420939'dur.
- Sayfalama, sayfa tablosu yapıları (hiyerarşik, hash, ters), Intel ve ARM mimarileri bu derste işlenmedi. Hoca bu kısımları da finale dahil etti, kitaptan çalışılmalı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.